

ESTUDIO — Carpeta dto/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/dto/` — 30 DTOs de request y 32 de response.

Qué es un DTO y por qué existe

DTO = *Data Transfer Object*. En criollo: una clase que existe solo para llevar datos de un lado al otro, sin lógica adentro.

Hay dos familias:

- **Request DTO** — lo que el frontend le manda al backend. Vive en `dto/request/` .
- **Response DTO** — lo que el backend le devuelve al frontend. Vive en `dto/response/` .

La pregunta de mesa: "¿Por qué no devolvés directamente la entidad?"

Es la pregunta sobre DTOs. Cinco razones, en orden de importancia:

1. **Seguridad.** La entidad `Usuario` tiene el campo `password` con el hash adentro. Si devolvés la entidad, mandás el hash al navegador. Con un DTO elegís qué campos salen y cuáles no.
2. **Distinto público, distinto detalle.** El mismo comercio se muestra de tres maneras distintas según quién pregunte:

DTO	Quién lo ve	Lleva DNI del representante
<code>ComercioPublicoResponseDTO</code>	Cualquiera, sin login	No
<code>ComercioResponseDTO</code>	El dueño viendo su propio perfil	Sí
<code>ComercioAdminResponseDTO</code>	El administrador	Sí, y además el email de la cuenta

Con una sola entidad no podrías hacer eso.

3. **Desacoplar la API de la base de datos.** Si mañana cambiás el nombre de una columna, cambiás el mapeo en el service y el frontend ni se entera. Sin DTO, cualquier cambio en la base rompe la API.
4. **Los datos no siempre coinciden 1 a 1.** `ProductoResponseDTO` tiene `nombreCategoria` y `nombreComercio` , que en la base no están en la tabla `producto` — están en otras tablas. El DTO los junta para que el frontend no tenga que hacer tres llamadas.
5. **Evitar el problema de la serialización infinita.** Si `Comercio` tiene una lista de `Producto` , y cada `Producto` apunta de vuelta a su `Comercio` , convertir eso a JSON directo entra en un loop infinito. El DTO corta esa cadena.

Las anotaciones que se repiten en todos los DTOs

Anotación (Lombok)	Qué hace
<code>@Getter</code>	Genera los <code>getX()</code> de todos los campos.
<code>@Setter</code>	Genera los <code>setX()</code> . Solo en los request — los response son inmutables.
<code>@NoArgsConstructor</code>	Constructor vacío. Jackson (el que convierte JSON ↔ Java) lo necesita.
<code>@AllArgsConstructor</code>	Constructor con todos los campos.

Diferencia estructural entre request y response, que vale la pena notar: los request tienen `@Setter` y campos normales (Jackson tiene que poder llenarlos leyendo el JSON). Los response tienen todos los campos `private final` y un constructor explícito — son **inmutables**: se arman una vez en el service y nadie los puede modificar después.

PARTE 1 — LOS 30 REQUEST DTO

Guía rápida de las validaciones estándar

Antes de la lista, el diccionario de las anotaciones de Bean Validation que vas a ver repetidas:

Anotación	Qué exige, en criollo
@NotBlank	No puede ser nulo, ni vacío, ni solo espacios. Para textos.
@NotNull	No puede ser nulo. Para números, fechas, enums, objetos.
@NotEmpty	Para listas: tiene que tener al menos un elemento.
@Size(max = N)	No más de N caracteres (o elementos, si es lista).
@Email	Tiene que tener forma de email.
@Pattern(regexp = "...")	Tiene que coincidir con una expresión regular.
@Min / @Max	Valor numérico mínimo / máximo.
@Positive	Mayor a cero.
@PositiveOrZero	Cero o más.
@Digits(integer=8, fraction=0)	Hasta 8 dígitos enteros y 0 decimales.
@Past / @PastOrPresent	Fecha que ya pasó (o es hoy).
@Valid	"Validá también lo de adentro" — para objetos y listas anidadas.

Las anotaciones custom (`@ValidarCuit` , `@ValidarFormatoDni` , etc.) están explicadas en detalle en `ESTUDIO-VALIDATION.md`.

1.1 Autenticación y cuenta (9 DTOs)

LoginRequestDTO — el body del login

Campo	Tipo	Validaciones
email	String	<code>@NotBlank</code> (no vacío), <code>@Email</code> (formato de mail)
password	String	<code>@NotBlank</code>

Fijate que el password NO tiene `@ValidarPasswordSegura` . Es a propósito: al loguearte no importa si tu contraseña es segura, importa si es **la correcta**. Validar la fortaleza acá sería absurdo — y peor, le daría pistas a un atacante sobre qué formato tienen las contraseñas válidas.

RegistroClienteRequestDTO — el formulario de registro de cliente

Es uno de los DTOs más grandes y más interesantes del proyecto.

Campo	Tipo	Validaciones
nombre	String	<code>@NotBlank</code> ("El nombre es obligatorio"), <code>@ValidarFormatoNombre</code> , <code>@Size(max=100)</code>
apellido	String	<code>@NotBlank</code> , <code>@ValidarFormatoNombre</code> ("solo puede contener letras"), <code>@Size(max=100)</code>
dni	String	<code>@NotBlank</code> , <code>@ValidarFormatoDni</code>
fechaNacimiento	LocalDate	<code>@NotNull</code> , <code>@ValidarFechaNacimientoPlausible</code>
telefono	String	<code>@NotBlank</code> , <code>@ValidarTelefonoArgentino</code> , <code>@Size(max=30)</code>
email	String	<code>@NotBlank</code> , <code>@ValidarFormatoEmail</code> , <code>@Size(max=254)</code>
password	String	<code>@NotBlank</code> , <code>@ValidarPasswordSegura</code>
direccion	DireccionRequestDTO	<code>@NotNull</code> , <code>@Valid</code> (valida la dirección por dentro)
fotoPerfilUrl	String	<code>@ValidarUrlCloudinary</code> , <code>@Size(max=500)</code> — opcional , no tiene <code>@NotBlank</code>

Lo interesante: los setters manuales de normalización

Este DTO no usa `@Setter` de Lombok en varios campos. Escribe los setters a mano para limpiar el dato antes de validarlo:

Setter	Qué limpia
<code>setNombre</code> / <code>setApellido</code>	Saca espacios de los bordes y colapsa espacios internos (<code>"Juan Carlos"</code> → <code>"Juan Carlos"</code>).
<code>setDni</code>	Saca puntos, guiones y espacios (<code>"12.345.678"</code> → <code>"12345678"</code>).
<code>setTelefono</code>	Saca espacios, paréntesis y guiones (<code>"(2964) 45-6789"</code> → <code>"2964456789"</code>).
<code>setEmail</code>	Trim y pasa todo a minúsculas (<code>" Juan@Mail.COM "</code> → <code>"juan@mail.com"</code>).

Por qué esto importa, en dos motivos:

1. **Mensajes de error correctos.** Si alguien escribe solo espacios en el nombre, sin normalización el `@NotBlank` pasaría (hay caracteres) y saltaría el error de formato: *"el nombre solo puede contener letras"*, que confunde. Normalizando primero, queda vacío y sale el mensaje correcto: *"el nombre es obligatorio"*.
2. **Lo que se guarda ya está limpio.** El email en minúsculas evita que `Juan@mail.com` y `juan@mail.com` se registren como dos cuentas distintas. El DNI sin puntos evita que `12.345.678` y `12345678` parezcan personas diferentes.

RegistroComercioRequestDTO — el DTO más grande del proyecto (24 campos)

Junta en un solo formulario los datos de: la **empresa** (PersonaJuridica), el **representante legal** (PersonaFisica), el **comercio** en sí, y las **credenciales de acceso**.

Datos fiscales / legales de la empresa:

Campo	Tipo	Validaciones
razonSocial	String	@NotBlank , @Pattern (no puede ser solo símbolos), @Size(max=150)
cuit	String	@NotBlank , @ValidarCuit . Setter manual: saca todo lo que no sea dígito
condicionIva	CondicionIva (enum)	@NotNull
tipoSociedad	TipoPersonaJuridica (enum)	@NotNull
domicilioFiscal	String	@NotBlank , @Pattern , @Size(max=255)
fechaInicioActividades	LocalDate	@NotNull , @PastOrPresent (no puede ser futura)

Datos del comercio (lo que ve el público):

Campo	Tipo	Validaciones
nombre	String	@NotBlank , @Pattern , @Size(max=150)
descripcion	String	@Size(max=2000) — opcional
telefono	String	@NotBlank , @ValidarTelefonoArgentino , @Size(max=30)
emailContacto	String	@NotBlank , @Email , @Size(max=150)
tipoComercio	TipoComercio (enum)	@NotNull
aceptaDelivery	boolean	Ninguna (es primitivo, no puede ser nulo)
aceptaRetiro	boolean	Ninguna
fotoPerfilUrl	String	@NotBlank ("Agregá una foto de perfil"), @ValidarUrlCloudinary , @Size(max=500) — obligatorio

Credenciales de acceso:

Campo	Tipo	Validaciones
email	String	@NotBlank , @ValidarFormatoEmail , @Size(max=254) . Setter manual: trim + minúsculas
password	String	@NotBlank , @ValidarPasswordSegura

Bloques anidados:

Campo	Tipo	Validaciones
direccion	DireccionRequestDTO	@NotNull , @Valid
horarios	List<HorarioRequestDTO>	@NotEmpty (al menos una franja), @Valid
redesSociales	List<RedSocialRequestDTO>	@NotEmpty (al menos una), @Size(max=5) , @Valid

Datos del representante legal (los 5 campos):

Campo	Tipo	Validaciones
nombreRepresentante	String	@NotBlank , @ValidarFormatoNombre , @Size(max=100)
apellidoRepresentante	String	@NotBlank , @ValidarFormatoNombre , @Size(max=100)
dniRepresentante	String	@NotBlank , @ValidarFormatoDni
telefonoRepresentante	String	@NotBlank , @ValidarTelefonoArgentino , @Size(max=30)
fechaNacimientoRepresentante	LocalDate	@NotNull , @MayorDeEdad

Cuatro cosas de este DTO que son excelente material de mesa

1. Hay DOS emails distintos, a propósito. `email` es con el que te logueás; `emailContacto` es el que se muestra en la pantalla pública del comercio. Pueden ser distintos: el dueño se loguea con su mail personal, pero el público ve `info@pizzeria.com` .

2. El representante **Sí** tiene `@MayorDeEdad` , pero el cliente **NO**. Esa diferencia es intencional, no un descuido de sincronización: `RegistroClienteRequestDTO.fechaNacimiento` usa `@ValidarFechaNacimientoPlausible` (una fecha razonable de nacimiento), mientras que el representante legal de una empresa **sí debe ser mayor de 18** por una razón de negocio real.

3. Al menos una red social es obligatoria. El `@NotEmpty` en `redesSociales` es una regla de negocio del proyecto: un comercio tiene que tener al menos una forma de contacto en redes.

4. Reutiliza `RedSocialRequestDTO` tal cual. No se creó un DTO nuevo para "red social dentro del registro": el body del alta individual y el del registro tienen exactamente la misma forma, así que se reutilizó. Menos código duplicado, una sola definición de las validaciones.

Los DTOs del flujo de contraseña y verificación (6 DTOs, casi idénticos)

DTO	Campos	Validaciones
RecuperacionPasswordRequestDTO	email	@NotBlank , @Email
ReactivacionCuentaRequestDTO	email	@NotBlank , @Email
ReenviarVerificacionRequestDTO	email	@NotBlank , @Email
VerificarCodigoRequestDTO	email , codigo	email: @NotBlank + @Email ; código: @NotBlank + @Pattern("\\d{6}")
ValidarCodigoRecuperacionRequestDTO	email , codigo	Idénticas al anterior
ConfirmarReactivacionCuentaRequestDTO	email , codigo	Idénticas
ConfirmarRecuperacionPasswordRequestDTO	email , codigo , nuevaPassword	+ @ValidarPasswordSegura en la contraseña

El `@Pattern(regexp = "\\d{6}")` significa **exactamente 6 dígitos numéricos**. `\\d` es "un dígito", `{6}` es "seis veces". Nada de letras, nada de 5 ni 7 caracteres.

Por qué tres DTOs casi iguales en vez de uno solo: porque son operaciones distintas, y si mañana una necesita un campo extra, no arrastrás a las otras. Es un principio de diseño de API: cada endpoint tiene su propio contrato explícito.

CambioPasswordPerfilRequestDTO — cambiar contraseña estando logueado

Campo	Tipo	Validaciones
passwordActual	String	@NotBlank
passwordNueva	String	@NotBlank , @ValidarPasswordSegura

Por qué pide la contraseña actual: si alguien te deja la sesión abierta en una compu ajena, sin esta verificación cualquiera podría cambiarte la contraseña y quedarse con la cuenta. Otra vez, la nueva sí se valida por fortaleza, la actual no.

1.2 Dirección, horario y ubicación (2 DTOs)

DireccionRequestDTO

Se reutiliza dentro de los dos registros (cliente y comercio).

Campo	Tipo	Validaciones
calle	String	@NotBlank , @Size(max=150) , @Pattern (no puede ser solo símbolos)
numero	String	@NotBlank , @Size(max=10) , @Pattern("^\\d+\$") — solo números
pisoDepto	String	@Size(max=30) , @Pattern que acepta vacío o algo con letras/números — opcional
codigoPostal	String	@NotBlank , @ValidarCodigoPostalArgentino
localidadId	String	@NotBlank ("Seleccioná tu localidad"), @Size(max=15)
principal	boolean	Ninguna

Detalle de diseño: este DTO **no tiene** `clienteId` **ni** `comercioId` . A quién pertenece la dirección lo decide el flujo que la contiene: si venís por el registro de cliente, se asocia al cliente; si venís por el de comercio, al comercio. **La exclusión mutua queda garantizada por tener dos flujos separados**, no por una validación. Es más robusto que un `if` — no hay forma de mandar las dos cosas porque el campo directamente no existe.

Por qué `localidadId` **es** `String` **y no** `Integer`: los ids vienen de la API Georef del gobierno, y son códigos como `"94008"` , no números autoincrementales.

HorarioRequestDTO

Campo	Tipo	Validaciones
diaSemana	DiaSemana (enum)	@NotNull
horaApertura	LocalTime	@NotNull
horaCierre	LocalTime	@NotNull

Ojo con esto: que la hora de cierre sea posterior a la de apertura **no se valida acá**, se valida en el `ComercioService` . ¿Por qué? Porque Bean Validation valida **un campo a la vez**; comparar dos campos entre sí requiere lógica que ve el objeto completo, y eso es responsabilidad del service.

Cómo se modela el horario: hay **0 o más filas por día**. Un día sin ninguna fila = cerrado ese día (no hace falta un booleano "abre/cierra"). Horario partido (mañana y tarde) = dos filas del mismo día.

1.3 Producto y catálogo (5 DTOs)

ProductoRequestDTO

Campo	Tipo	Validaciones
nombre	String	@NotBlank , @Pattern (letras, números y espacios), @Size(max=150)
descripcion	String	@Size(max=2000) — opcional
precio	BigDecimal	@NotNull , @Positive ("mayor a \$0"), @Digits(integer=8, fraction=0)
categoriaId	Integer	@NotNull
tagIds	List<Integer>	@Size(max=5) ("No podés seleccionar más de 5 tags")

Tres cosas para destacar:

- El **precio** es `BigDecimal` , no `double` . Pregunta clásica: "*¿por qué?*". Porque `double` usa punto flotante binario y **no puede representar exactamente los decimales**: en Java, `0.1 + 0.2` da `0.30000000000000004` . Con plata eso es inaceptable — los centavos se te van corriendo. `BigDecimal` guarda el número en decimal exacto. **Para dinero siempre `BigDecimal` .**
 - `fraction = 0` **significa que no se aceptan centavos**. Decisión de negocio: los precios en Río Grande van en pesos enteros.
 - Lo que **NO** está en el DTO es tan importante como lo que está:
- No hay `estado`** — el producto nace `DISPONIBLE` por decisión del service, y se cambia con un endpoint dedicado (`PATCH /productos/{id}/estado`).
 - No hay imágenes** — la galería tiene su propio flujo con firma de Cloudinary.
 - Sí hay `tagIds`** — porque la relación producto↔tag no tiene endpoint propio, se maneja como parte del alta/edición del producto.

CambioEstadoProductoRequestDTO

Campo	Tipo	Validaciones
estado	EstadoProducto (enum)	@NotNull

Un solo campo. Al ser un **enum**, si mandás un valor que no existe, Jackson falla al deserializar y devuelve 400 solo. **El tipo mismo es la validación** — no hace falta un `@Pattern` con los valores posibles.

ImagenProductoRequestDTO

Campo	Tipo	Validaciones
url	String	@NotBlank , @ValidarUrlCloudinary , @Pattern que exige terminar en <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> O <code>.webp</code>
orden	Integer	@NotNull , @PositiveOrZero
esPrincipal	boolean	Ninguna

El `(?i)` al principio del `@Pattern` significa **case-insensitive**: acepta `.JPG` igual que `.jpg` .

OrdenImagenRequestDTO y UrlImagenRequestDTO

DTO	Campo	Validaciones
OrdenImagenRequestDTO	orden (Integer)	@NotNull , @PositiveOrZero
UrlImagenRequestDTO	url (String)	@NotBlank , @ValidarUrlCloudinary

Dos DTOs de un solo campo. Uno podría preguntarse por qué no mandar el número pelado. La razón es que **un JSON siempre tiene que ser un objeto** (`{ "orden": 3 }`), no un valor suelto — así, si mañana hace falta agregar un campo, no rompés el contrato del endpoint.

1.4 Carrito y pedido (3 DTOs)

ItemCarritoRequestDTO

Campo	Tipo	Validaciones
productoId	Integer	@NotNull ("El producto es obligatorio")
cantidad	Integer	@NotNull , @Min(1) ("La cantidad mínima es 1"), @Max(20) ("La máxima es 20")
nota	String	@Size(max=255) — opcional ("sin cebolla", "bien cocida")

El tope de 20 es una regla de negocio: nadie pide 500 hamburguesas por la app, y evita que alguien haga un pedido absurdo por error o a propósito.

ActualizarCantidadItemCarritoRequestDTO

Campo	Tipo	Validaciones
cantidad	Integer	@NotNull , @Min(1) , @Max(20)

Las mismas reglas de cantidad que el anterior, para que sea imposible saltar el límite editando en vez de agregando.

PedidoRequestDTO

Campo	Tipo	Validaciones
tipoEntrega	TipoEntrega (enum)	@NotNull ("Debés seleccionar una modalidad de entrega")
direccionId	Integer	Ninguna

Por qué `direccionId` no tiene `@NotNull` , y es la parte interesante: es **condicionalmente obligatorio**. Si elegís `DOMICILIO` , hace falta; si elegís `RETIRO` , no. Bean Validation no puede expresar "obligatorio solo si otro campo vale X" sin una anotación custom a nivel de clase.

La decisión fue validarlo en `PedidoService` , porque es **una regla de negocio**, no una regla de formato. Y es un solo caso — no valía la pena crear una anotación custom para usarla una vez.

Fijate lo que este DTO NO tiene: no tiene la lista de productos, ni el total. Porque el pedido se arma **a partir del carrito que ya está en la base**. El frontend no puede mandar los precios — si pudiera, alguien podría mandar `precio: 1` y comprar una pizza por un peso. Los precios los pone el backend leyendo la base.

1.5 Perfil y edición (3 DTOs)

ClienteEditarPerfilRequestDTO

Campo	Tipo	Validaciones
nombre	String	@NotBlank , @ValidarNombrePropio , @Size(max=100)
apellido	String	@NotBlank , @ValidarNombrePropio , @Size(max=100)
telefono	String	@NotBlank , @ValidarTelefonoArgentino , @Size(max=30)

Solo 3 campos. No están `email` , `dni` ni `fechaNacimiento` — son de solo lectura. Y no porque el service los ignore: **el DTO ni siquiera los tiene**, así que si los mandás, Jackson los tira a la basura al armar el objeto. Es una defensa estructural, mucho más fuerte que un `if` .

ComercioPerfilRequestDTO

Campo	Tipo	Validaciones
nombre	String	@NotBlank , @Pattern , @Size(max=150)
descripcion	String	@Size(max=2000)
telefono	String	@NotBlank , @ValidarTelefonoArgentino , @Size(max=30)
emailContacto	String	@NotBlank , @ValidarFormatoEmail , @Size(max=150)
aceptaDelivery	boolean	Ninguna
aceptaRetiro	boolean	Ninguna

Mismo criterio: no incluye `razonSocial`, `cuit` ni `condicionIva`. Este endpoint es para el perfil **público** del comercio, no para editar sus datos legales — cambiar un CUIT debería tener un proceso con verificación, no un formulario libre.

Tampoco incluye `fotoPerfilUr1`, y el motivo es sutil e interesante: `@ValidarUrlCloudinary` valida el **dominio**, no la **propiedad**. Un comercio podría pegar la URL de una foto subida por otro comercio. Por eso la foto tiene su propio flujo con firma (`POST /perfil/foto/firma` + `PUT /perfil/foto`), donde la firma está atada al `comercioId` del token y garantiza que la imagen es tuya.

FotoPerfilComercioRequestDTO y FotoPerfilUsuarioRequestDTO

Ambos, idénticos:

Campo	Tipo	Validaciones
url	String	<code>@NotBlank</code> , <code>@ValidarUrlCloudinary</code> , <code>@Pattern (jpg/jpeg/png/webp)</code> , <code>@Size(max=500)</code>

Son dos clases separadas aunque tengan la misma forma, porque son operaciones distintas (foto del comercio vs foto de la persona) y podrían divergir en el futuro.

1.6 Administración (4 DTOs)

CategoriaRequestDTO y TagRequestDTO

Ambos idénticos:

Campo	Tipo	Validaciones
nombre	String	<code>@NotBlank</code> , <code>@Size(max=100)</code>

AprobacionComercioRequestDTO

Campo	Tipo	Validaciones
aprobar	Boolean	<code>@NotNull</code>
motivo	String	<code>@Size(max=500)</code>

Por qué `motivo` es texto libre y no un enum: porque el modelo de datos no define ningún enum de motivos de rechazo de comercio (a diferencia del rechazo de **pedido**, que sí tiene `MotivoRechazo`). Se prefirió ser fiel al modelo antes que inventar una estructura que no existe.

Y el `motivo` es obligatorio si `aprobar` = `false`, pero eso se valida en el service, no acá — mismo caso que `direccionId` del pedido: es una regla condicional entre dos campos.

Fijate también que `aprobar` es `Boolean` (objeto) y no `boolean` (primitivo). Es a propósito: un `boolean` primitivo nunca puede ser nulo, así que si el frontend se olvida de mandarlo, valdría `false` en silencio y **rechazaría el comercio sin querer**. Con `Boolean` + `@NotNull`, el olvido salta como error 400.

RechazoPedidoRequestDTO

Campo	Tipo	Validaciones
motivo	<code>MotivoRechazo</code> (enum)	<code>@NotNull</code>
comentario	String	<code>@Size(max=500)</code> — opcional

Acá sí es un enum, porque los motivos de rechazo de pedido están definidos en el modelo de datos. El comentario libre es para agregar detalle.

RedSocialRequestDTO

Campo	Tipo	Validaciones
tipo	<code>TipoRedSocial</code> (enum)	<code>@NotNull</code> ("Seleccioná el tipo de red social")
url	String	<code>@NotBlank</code> , <code>@Pattern</code> (tiene que tener al menos una letra y un punto, sin espacios), <code>@Size(max=500)</code>

Tiene un setter manual muy práctico:

```
public void setUrl(String url) {
    this.url = TextoUtils.normalizarUrlConEsquema(url);
}
```

Si el usuario escribe `instagram.com/mipizzeria`, el setter le agrega el `https://` solo. Es un detalle de usabilidad: nadie escribe el `https://` a mano. La lógica está en `TextoUtils`, ver `ESTUDIO-UTIL.md`.

PARTE 2 — LOS 32 RESPONSE DTO

Los response no tienen validaciones (nadie valida lo que uno mismo genera). Lo que importa es **qué campos exponen y por qué**.

2.1 El envoltorio

ApiResponse<T> — el más importante de todos

Campo	Tipo	Qué es
mensaje	String	Un texto legible para el usuario.
data	T (genérico)	El contenido real, del tipo que sea.

Todas las respuestas de la API tienen esta forma:

```
{ "mensaje": "Producto creado correctamente", "data": { "id": 42, "nombre": "Pizza" } }
```

El `<T>` es un **genérico**: la misma clase sirve para devolver un producto, una lista de pedidos, un número, o nada (`ApiResponse<Void>`). Es lo que evita tener 32 clases envoltorio distintas.

El **status HTTP** (200, 201, 404...) no va acá adentro — lo maneja el `ResponseEntity` por fuera.

2.2 Usuario y autenticación

LoginResponseDTO

Campo	Tipo	Qué es
token	String	El JWT.
usuario	UsuarioResponseDTO	Los datos básicos del que se logueó.

El frontend guarda el token (en `localStorage`) y lo manda en cada llamada siguiente.

UsuarioResponseDTO

Campo	Tipo	Qué es
id	Integer	Id del usuario.
email	String	Su email.
rol	RolUsuario	CLIENTE / DUENO / ADMINISTRADOR.
estado	EstadoUsuario	ACTIVO / PENDIENTE / BLOQUEADO / etc.
fotoPerfilUrl	String	La URL de su foto.

Lo importante es lo que **NO** está: el `password` . La entidad `usuario` lo tiene, este DTO no. Ese es exactamente el motivo de existir de los DTOs, en un solo ejemplo.

AdministradorResponseDTO

Campo	Tipo	Qué es
id , nombre , apellido , fotoPerfilUrl		Lo mínimo para el header del panel de admin.

2.3 Cliente

ClienteResponseDTO — el perfil propio

Campo	Tipo
id	Integer
nombre , apellido , dni	String
fechaNacimiento	LocalDate
telefono , email	String
direccion	DireccionResponseDTO (anidado)
fotoPerfilUrl	String

Devuelve más campos de los que se pueden editar: el DNI y la fecha se muestran pero son de solo lectura (el DTO de edición no los tiene).

ClienteAdminResponseDTO — el listado que ve el admin

Campo	Tipo
id , nombre , apellido , dni , email	
estado	EstadoUsuario
fechaRegistro	LocalDateTime

Diferencia con el anterior: el admin ve el estado y la fechaRegistro (le sirven para gestionar), pero no ve la dirección ni el teléfono (no los necesita para su tarea). Otra vez el mismo dato, distinto recorte según el público.

2.4 Los tres DTOs de Comercio — el ejemplo estrella

Este es el mejor ejemplo del proyecto para explicar para qué sirven los DTOs. Comparalos:

Campo	ComercioPublicoResponseDTO	ComercioResponseDTO	ComercioAdminResponseDTO
id , nombre , descripcion , fotoPerfilUrl	Sí	Sí	Sí
telefono , emailContacto , tipoComercio	Sí	Sí	Sí
aceptaDelivery , aceptaRetiro , estado	Sí	Sí	Sí
razonSocial , cuit	Sí	Sí	Sí
direccion , horarios	Sí	Sí	Sí
condicionIva	No	Sí	Sí
tipoSociedad , domicilioFiscal , fechaInicioActividades	No	Sí	No
representante (con DNI)	No	Sí	Sí
motivoRechazo	No	Sí	No
emailCuenta (el de login)	No	No	Sí
fechaRegistro	No	No	Sí
redesSociales	No	No	Sí

Cómo contarlo en la mesa:

- El público ve lo que necesita para decidir si pide comida ahí: nombre, foto, teléfono, horarios, si hace delivery. Nunca el DNI del representante — sería una filtración de dato personal sin autenticación.
- El dueño ve su ficha completa, incluidos sus datos fiscales y —si lo rechazaron— el motivo.
- El admin ve lo que necesita para aprobar: los datos fiscales clave, quién es el representante, cuándo se registró y con qué email entra.

RepresentanteResponseDTO (anidado)

Campo	Tipo
nombre , apellido , dni , telefono	String
fechaNacimiento	LocalDate

Va anidado dentro de ComercioResponseDTO y ComercioAdminResponseDTO , y nunca dentro de ComercioPublicoResponseDTO . Al estar en una clase aparte, es imposible que se cuele por error en el DTO público — para incluirlo hay que declararlo explícitamente.

2.5 Producto y catálogo

ProductoResponseDTO

Campo	Tipo	Qué es
id	Integer	Id del producto.
nombre , descripcion	String	
precio	BigDecimal	
categoriaId	Integer	Id de la categoría.
nombreCategoria	String	El nombre, ya resuelto.
comercioId	Integer	Id del comercio.
nombreComercio	String	El nombre, ya resuelto.
estado	EstadoProducto	
imagenes	List<ImagenProductoResponseDTO>	La galería.
tags	List<String>	Los nombres de los tags.

Detalle de diseño para destacar: manda el **id** y el **nombre** de la categoría y del comercio. Sin el nombre, el frontend tendría que hacer una llamada extra por cada producto solo para mostrar "Pizzas" en vez de "categoría 3". Es un ejemplo concreto de cómo el DTO **junta datos de varias tablas** para ahorrarle trabajo al cliente.

Y los `tags` son `List<String>` (solo los nombres), no objetos completos — porque para mostrarlos alcanza con el texto.

ImagenProductoResponseDTO

Campo	Tipo
id , orden	Integer
url	String
esPrincipal	boolean

ProductosPaginadosResponseDTO

Campo	Tipo	Qué es
productos	List<ProductoResponseDTO>	Los de esta página.
paginaActual	int	En qué página estás.
totalPaginas	int	Cuántas hay.
totalProductos	long	El total sin paginar.

Por qué hace falta: si devolvieras solo la lista, el frontend no podría dibujar el paginador — no sabría si hay más páginas. Los tres números extra son metadatos de la paginación.

CategoriaResponseDTO y TagResponseDTO

Ambos con la misma forma:

Campo	Tipo	Qué es
id	Integer	
nombre	String	
activo	boolean	Si está dado de baja lógicamente o no.
cantidadProductos	long	Cuántos productos la usan.

`cantidadProductos` es un dato calculado, no una columna de la base. Se cuenta con `ProductoRepository.countByCategoriaId` / `ProductoTagRepository.countByTagId` . Sirve para que en el panel de admin se lea "Pizzas — 12 productos asociados" y el admin sepa el impacto antes de dar de baja algo.

CategoriaFiltroResponseDTO , TagFiltroResponseDTO y FiltrosCatalogoResponseDTO

DTO	Campos
CategoriaFiltroResponseDTO	id , nombre
TagFiltroResponseDTO	id , nombre
FiltrosCatalogoResponseDTO	categorias (lista del primero) + tags (lista del segundo)

Versiones ultra-livianas para los chips de filtro del catálogo público. No mandan `activo` ni `cantidadProductos` porque para pintar un botón de filtro no hacen falta — y menos datos por la red es más rápido.

2.6 Carrito y pedido

CarritoResponseDTO

Campo	Tipo	Qué es
<code>comercioId</code>	<code>Integer</code>	El comercio del carrito. <code>null</code> si está vacío.
<code>nombreComercio</code>	<code>String</code>	Su nombre. <code>null</code> si está vacío.
<code>items</code>	<code>List<ItemCarritoResponseDTO></code>	Los productos.
<code>subtotal</code>	<code>BigDecimal</code>	La suma.

No tiene `id` propio, y es intencional: el carrito es 1 a 1 con el cliente, y ninguna operación necesita referenciarlo por `id` — todas actúan sobre "el carrito del cliente actual".

El `comercioId` en null cuando está vacío refleja una regla de negocio clave: **el carrito solo puede tener productos de un comercio a la vez**. Cuando se vacía, se "desata" del comercio y podés empezar en otro.

ItemCarritoResponseDTO

Campo	Tipo
<code>id</code> , <code>productoId</code> , <code>cantidad</code>	<code>Integer</code>
<code>nombreProducto</code> , <code>nota</code>	<code>String</code>
<code>precioUnitario</code> , <code>subtotal</code>	<code>BigDecimal</code>

El `subtotal` viene ya calculado del backend. **Nunca se le confía el cálculo al frontend** — podrían manipularlo desde el navegador.

PedidoResponseDTO

Campo	Tipo	Qué es
<code>id</code>	<code>Integer</code>	Nº de pedido.
<code>clienteId</code>	<code>Integer</code>	Quién lo hizo.
<code>nombreCliente</code>	<code>String</code>	Su nombre (para que el comercio lo vea).
<code>comercioId</code>	<code>Integer</code>	A quién.
<code>estado</code>	<code>EstadoPedido</code>	En qué anda.
<code>tipoEntrega</code>	<code>TipoEntrega</code>	Delivery o retiro.
<code>direccion</code>	<code>DireccionResponseDTO</code>	Dónde entregarlo (null si es retiro).
<code>motivoRechazo</code>	<code>MotivoRechazo</code>	Si lo rechazaron, por qué.
<code>comentarioRechazo</code>	<code>String</code>	El detalle libre del rechazo.
<code>fechaCreacion</code>	<code>LocalDateTime</code>	Cuándo se hizo.
<code>detalles</code>	<code>List<DetallePedidoResponseDTO></code>	Los productos.
<code>total</code>	<code>BigDecimal</code>	El total.

`nombreCliente` está porque **no existe un catálogo público de clientes**: el comercio no tendría forma de resolver ese nombre desde el frontend, así que el backend lo manda ya resuelto.

DetallePedidoResponseDTO

Campo	Tipo
<code>productoId</code> , <code>cantidad</code>	<code>Integer</code>
<code>nombreProducto</code> , <code>nota</code>	<code>String</code>
<code>precioUnitario</code> , <code>subtotal</code>	<code>BigDecimal</code>

Concepto importante para la mesa: el precio es un "snapshot". Cuando confirmás un pedido, el precio del producto se **copia** al detalle. Si mañana el comercio sube el precio de la pizza, tu pedido de ayer sigue mostrando lo que pagaste. **Un pedido es un documento histórico, inmutable**. Si el detalle apuntara al precio actual del producto, el historial mentiría.

ResumenPedidosHoyResponseDTO

Campo	Tipo	Qué es
totalFacturadoHoy	BigDecimal	Cuánta plata entró hoy.
cantidadPedidosHoy	int	Cuántos pedidos.
cantidadPendientes	int	Cuántos esperando respuesta.

Los tres números del dashboard del comercio.

2.7 Notificación, dirección, geografía y otros

NotificacionResponseDTO

Campo	Tipo	Qué es
id	Integer	
mensaje	String	El texto.
leida	boolean	Si ya la vio.
fechaCreacion	LocalDateTime	
entidadTipo	TipoEntidadNotificacion	A qué se refiere (un pedido, un comercio...).
entidadId	Integer	El id de esa cosa.

El par `entidadTipo` + `entidadId` es un patrón que vale la pena entender. Antes había un campo `pedidoId` , pero eso solo servía para notificaciones de pedido. Con este par, una notificación puede apuntar a **cualquier** entidad: `entidadTipo = PEDIDO`, `entidadId = 42` , O `entidadTipo = COMERCIO`, `entidadId = 7` . Es lo que le permite al frontend saber a qué pantalla llevarte cuando hacés clic.

Se llama **referencia polimórfica**. El precio de la flexibilidad es que **la base no puede garantizar la integridad con una FK** — no hay forma de que MySQL sepa a qué tabla apunta. Ese control queda del lado de la aplicación.

DireccionResponseDTO

Campo	Tipo
id	Integer
calle , numero , pisoDepto , codigoPostal	String
localidadId	String
nombreLocalidad , nombreProvincia	String
principal	boolean

Otra vez: manda **los nombres ya resueltos**. Sin eso, el frontend recibiría `localidadId: "94008"` y tendría que hacer dos llamadas más para mostrar "Río Grande, Tierra del Fuego".

ProvinciaResponseDTO y LocalidadResponseDTO

DTO	Campos
ProvinciaResponseDTO	<code>id</code> (<code>String</code>), <code>nombre</code> (<code>String</code>)
LocalidadResponseDTO	<code>id</code> , <code>nombre</code> , <code>provinciaId</code> (los 3 <code>String</code>)

Los ids son `String` porque vienen de la API Georef del gobierno.

HorarioResponseDTO

Campo	Tipo
id	Integer
diaSemana	DiaSemana
horaApertura , horaCierre	LocalTime

RedSocialResponseDTO

Campo	Tipo
id	Integer
tipo	TipoRedSocial
url	String
fechaCreacion , fechaModificacion	LocalDateTime

CloudinarySignatureResponseDTO

Campo	Tipo	Qué es
signature	String	La firma criptográfica.
timestamp	long	Cuándo se generó (la firma caduca).
apiKey	String	La clave pública de Cloudinary.
cloudName	String	La cuenta.
folder	String	En qué carpeta se guarda.
uploadPreset	String	La configuración de subida.

Pregunta razonable: "¿No es peligroso mandarle la `apiKey` al navegador?" No: la `apiKey` es la **pública**, está pensada para eso. Lo que nunca sale del servidor es el `apiSecret` , que es lo que se usa para generar la firma. Sin el secret no podés fabricar firmas nuevas, y la que te dieron está limitada a una carpeta específica y caduca.

MetricasAdminResponseDTO

Campo	Tipo
comerciosPendientes , comerciosTotal , clientesTotal , categoriasActivas , tagsActivos	long

Los 5 números del dashboard del admin. Son **conteos agregados** (`COUNT(*)`), no listados — es mucho más barato que traer todas las filas para contarlas del lado de Java.

Preguntas típicas de mesa sobre DTOs

"¿Qué es un DTO?" Un objeto que solo transporta datos entre capas, sin lógica. Los de request son lo que entra, los de response lo que sale.

"¿Por qué no usás las entidades directamente?" Por seguridad (la entidad `usuario` tiene el hash de la contraseña), por poder mostrar el mismo dato con distinto detalle según el rol, y para que un cambio en la base no rompa la API.

"¿Dónde están las validaciones?" Como anotaciones en los campos de los request DTO. Se disparan con `@Valid` en el controller, antes de que la petición entre al método.

"Mostrame un ejemplo de por qué los DTOs importan." Comercio tiene tres DTOs de response. El público no lleva el DNI del representante; el del propio dueño y el del admin sí. Con una sola entidad no habría forma de hacer esa distinción.

"¿Por qué el precio es `BigDecimal`?" Porque `double` no representa decimales exactos y con plata eso genera errores de centavos. `BigDecimal` es exacto.

"¿Por qué algunas validaciones están en el service y no en el DTO?" Porque Bean Validation valida un campo a la vez. Las reglas que dependen de otro campo (que `direccionId` sea obligatorio solo si el tipo de entrega es domicilio, que la hora de cierre sea posterior a la de apertura) o que necesitan consultar la base (que el email no esté repetido) van en el service.

Índice de DTOs cubiertos en este documento

Request (30)

DTO	Para qué
LoginRequestDTO	Body del login.
RegistroClienteRequestDTO	Registro de cliente.
RegistroComercioRequestDTO	Registro de comercio (el más grande, 24 campos).
RecuperacionPasswordRequestDTO	Pedir código de recuperación.
ValidarCodigoRecuperacionRequestDTO	Validar ese código.
ConfirmarRecuperacionPasswordRequestDTO	Fijar la contraseña nueva.
ReactivacionCuentaRequestDTO	Pedir código de reactivación.
ConfirmarReactivacionCuentaRequestDTO	Confirmar la reactivación.
VerificarCodigoRequestDTO	Verificar la cuenta con código de 6 dígitos.
ReenviarVerificacionRequestDTO	Reenviar el código de verificación.
CambioPasswordPerfilRequestDTO	Cambiar contraseña estando logueado.
DireccionRequestDTO	Bloque de dirección, reutilizado en los dos registros.
HorarioRequestDTO	Una franja horaria del comercio.
ProductoRequestDTO	Alta y edición de producto.
CambioEstadoProductoRequestDTO	Cambiar el estado de un producto.
ImagenProductoRequestDTO	Registrar una imagen de la galería.
OrdenImagenRequestDTO	Reordenar una imagen.
UrlImagenRequestDTO	Reemplazar la URL de una imagen.
ItemCarritoRequestDTO	Agregar un producto al carrito.
ActualizarCantidadItemCarritoRequestDTO	Cambiar la cantidad de un ítem.
PedidoRequestDTO	Confirmar el pedido.
RechazoPedidoRequestDTO	Rechazar un pedido, con motivo.
AprobacionComercioRequestDTO	Aprobar o rechazar un comercio.
ClienteEditarPerfilRequestDTO	Editar perfil de cliente (3 campos).
ComercioPerfilRequestDTO	Editar perfil público del comercio.
FotoPerfilComercioRequestDTO	URL de la foto del comercio.
FotoPerfilUsuarioRequestDTO	URL de la foto del usuario.
CategoriaRequestDTO	Crear/editar categoría.
TagRequestDTO	Crear/editar tag.
RedSocialRequestDTO	Crear/editar red social (con normalización de URL).

Response (32)

DTO	Para qué
ApiResponse<T>	El envoltorio de toda respuesta.
LoginResponseDTO	Token + datos del usuario.
UsuarioResponseDTO	Datos del usuario, sin el password.
AdministradorResponseDTO	Perfil del admin para el header.
CienteResponseDTO	Perfil propio del cliente.
CienteAdminResponseDTO	Listado de clientes para el admin.
ComercioPublicoResponseDTO	Comercio para el público, sin DNI.
ComercioResponseDTO	Comercio para su propio dueño.
ComercioAdminResponseDTO	Comercio para el administrador.
RepresentanteResponseDTO	Datos del representante legal (anidado).
ProductoResponseDTO	Producto con nombres de categoría y comercio resueltos.
ImagenProductoResponseDTO	Una imagen de la galería.
ProductosPaginadosResponseDTO	Productos + metadatos de paginación.
CategoriaResponseDTO	Categoría con su conteo de productos.
TagResponseDTO	Tag con su conteo de productos.
CategoriaFiltroResponseDTO	Categoría liviana para los chips de filtro.
TagFiltroResponseDTO	Tag liviano para los chips de filtro.
FiltrosCatalogoResponseDTO	Categorías + tags disponibles para filtrar.
CarritoResponseDTO	Carrito completo con subtotal.
ItemCarritoResponseDTO	Una línea del carrito.
PedidoResponseDTO	Pedido completo.
DetallePedidoResponseDTO	Una línea del pedido, con precio congelado.
ResumenPedidosHoyResponseDTO	Los 3 números del dashboard del comercio.
NotificacionResponseDTO	Notificación con referencia polimórfica.
DireccionResponseDTO	Dirección con localidad y provincia resueltas.
ProvinciaResponseDTO	Provincia (id, nombre).
LocalidadResponseDTO	Localidad (id, nombre, provinciald).
HorarioResponseDTO	Una franja horaria.
RedSocialResponseDTO	Una red social del comercio.
CloudinarySignatureResponseDTO	La firma de subida de imágenes.
MetricasAdminResponseDTO	Los 5 conteos del dashboard del admin.